Zephyr Project

IT-Schulungen.com
Technology & Transformation

Zephyr-Workspace und west

Quellen, Manifest und Workspace-Verwaltung

1.05

Vom Workspace zur reproduzierbaren Quelle

Fünf Perspektiven auf Zephyrs Multi-Repository-Struktur

01

Wurzel

Top-Level und Verzeichnisstruktur

02

Manifest

west.yml beschreibt Projekte und Revisionen

03

Projekte

Zephyr, Module und Manifest-Repository

04

Befehle

init, update, list und topdir

05

Stabilität

Kompatible, festgelegte Quellenstände

Ziel: Workspace-Wurzel, Manifest, Zephyr-Quellen und mindestens zwei Module sicher finden.

Anatomie eines Zephyr-Workspaces

VERZEICHNISSE

```
workspace/
├── .west/
├── zephyr/
├── modules/
├── bootloader/
└── application/
```

01 Top-Level

Gemeinsame Root aller durch west verwalteten Projekte.

02 Quellen

Zephyr und Module liegen als getrennte Git-Repositories vor.

03 Anwendung

Kann im Manifest-Projekt oder außerhalb von zephyr/ liegen.

Der Workspace ist eine koordinierte Sammlung von Repositories, kein einzelnes Repository.

Top-Level und lokale Konfiguration

TERMINAL

```
$ west topdir
/home/user/zephyr-workspace

$ ls -a $(west topdir)/.west
config
```

01 west topdir

Bestimmt die Workspace-Wurzel unabhängig vom aktuellen Unterverzeichnis.

02 .west/config

Speichert lokale Angaben zum Manifest und gehört nicht ins Projekt-Repository.

Die .west-Konfiguration verbindet den lokalen Workspace mit seinem Manifest-Projekt.

Das Manifest: west.yml

WEST.YML

```
manifest:
  remotes:
  projects:
    - name: zephyr
      revision: <festgelegt>
  self:
```

01 Manifest-Repository

Enthält west.yml und definiert die Zusammensetzung des Workspaces.

02 Festgelegte Revisionen

Jedes Projekt erhält URL, Pfad und kompatiblen Commit, Tag oder Branch.

west.yml ist die gemeinsame, versionierbare Quelle für Struktur und Revisionen.

Drei Rollen im Workspace

01	`Manifest-Projekt`	besitzt west.yml und steuert die Auswahl aller Projekte
02	`Zephyr-Repository`	liefert Kernel, Subsysteme, Treiber, Build-Logik und Samples
03	`Weitere Module`	ergänzen HALs, Bibliotheken, Bootloader, Tools und Hersteller-Code

Die Rollen sind getrennt; west hält ihre Revisionen und Pfade konsistent zusammen.

Die wichtigsten west-Befehle

WORKSPACE

west arbeitet relativ zur erkannten Workspace-Wurzel.

`west init`	legt einen neuen Workspace an und wählt das Manifest
`west update`	holt die im Manifest festgelegten Projektstände
`west list`	zeigt Namen, Pfade, URLs und Revisionen der Projekte
`west topdir`	gibt die zuverlässig erkannte Workspace-Wurzel aus
`west status`	macht lokale Änderungen in den Projekten sichtbar

Vor update zuerst Manifest und lokale Änderungen prüfen; danach das Ergebnis mit west list kontrollieren.

Warum festgelegte Revisionen zählen

`Manifest-Revision`	Versioniert die gewünschte Zusammensetzung des gesamten Workspaces
`Zephyr-Stand`	Bestimmt Kernel-APIs, Build-Logik und erwartete Modul-Schnittstellen
`Modulstände`	Passen zu dieser Zephyr-Version und werden gemeinsam aktualisiert
`west update`	Stellt die manifestierten Revisionen statt beliebiger aktueller Stände her
`Anti-Pattern`	Einzelne Repositories unabhängig auf main oder latest setzen

Reproduzierbarkeit entsteht durch gemeinsam festgelegte Revisionen, nicht durch möglichst neue Quellen.

Workspace-Verwaltung ist nicht der Build

WORKSPACE

Quellen koordinieren

west init und update

west list und topdir

Repositories und Revisionen

BUILD

Anwendung übersetzen

west build -b native_sim

CMake und Ninja

Konfiguration und Artefakte

SCHNITTSTELLE

Manifest → Build

Workspace liefert Quellen

Build wählt Anwendung und Ziel

Ausgabe liegt im Build-Verzeichnis

west bündelt beide Arbeitsbereiche, aber Workspace-Synchronisation und Build bleiben getrennte Vorgänge.

Geführte Workspace-Erkundung

FINDEN

Vier Nachweise

Workspace-Wurzel

Manifest west.yml

Zephyr-Repository

Mindestens zwei Module

ABLAUF

Gemeinsam prüfen

west topdir ausführen

west list lesen

Manifest lokalisieren

Pfade im Dateisystem öffnen

VERSTEHEN

Kurz erklären

Wer wählt Revisionen?

Was erzeugt west update?

Wo liegt .west?

Was gehört zum Build?

Lernkontrolle: Analysiere Root, Manifest, Zephyr und zwei Module im eigenen Workspace.